

# Akamata 詳細チュートリアル – Todo リスト API をゼロから作る

---

このチュートリアルは **Zig も Akamata も初めて**の人を対象に、空のディレクトリから始めて、最終的に **本番デプロイ可能な Todo リスト API + Web UI** を完成させます。

読了時間: 約 **60~90 分**。途中で休憩可能です。各章の冒頭に「この章のゴール」を書いてあるので、知っている部分はスキップしてください。

---

## 何を作るか

---

**Todo** リソースを管理する小さな Web アプリケーションです。

- `GET /api/todos` 未完了のものから順に Todo を一覧
- `POST /api/todos` Todo を作成 (タイトル必須、優先度オプション)
- `GET /api/todos/:id` 1 件取得
- `PUT /api/todos/:id` 更新 (タイトル変更、完了マーク)
- `DELETE /api/todos/:id` 削除
- `GET /health` 稼働確認
- `GET /metrics` Prometheus メトリクス
- `GET /` HTML UI (ブラウザで操作できる)

DB バックエンドは:

- **ローカル開発**: SQLite ファイル (`file:todo.db`)
- **本番デプロイ**: Cloudflare D1 (Workers にデプロイ)

最終的にこの 2 つの形態を **1 つのソースコード** から両方ビルド・デプロイします。

---

## 目次

---

0. 前提と環境セットアップ
1. プロジェクトの雛形を作る
2. 雛形を起動して中身を見る
3. Todo モデルを定義する
4. 自動マイグレーションで DB を作る
5. CRUD ハンドラを実装する
6. バリデーションを追加する
7. HTML UI を `embedFile` で同梱する
8. ルーティングとミドルウェアを整える

- 9. [Cloudflare Workers + D1 にデプロイする](#)
  - 10. [本番稼働のための観測 \(metrics + ログ\)](#)
  - 11. [よくある問題とデバッグ](#)
  - 12. [次のステップ](#)
- 

## 0. 前提と環境セットアップ

---

### この章のゴール

- 必要なツールが全部入っていることを確認
- Akamata の CLI が `akamata help` で動くことを確認

### 必須

- **Zig 0.16.0** — `zig version` で確認できる。なければ [ziglang.org/download](https://ziglang.org/download) から取得 (公式 tarball を `$HOME/.local` に展開して `PATH` を通すのが一番安全)
- macOS / Linux のターミナル (Windows は WSL2 推奨)
- `curl` (Linux/macOS 標準)

### あると便利 (Workers デプロイで必須)

- **Node.js 22 以降** + `npx wrangler` (Cloudflare Workers SDK)
- **Cloudflare アカウント** (無料プランで OK)
- ブラウザ (Chrome / Firefox / Safari)

### 確認

ターミナルで:

```
zig version
# 期待出力: 0.16.0 もしくは 0.16.0-dev.NNN+xxxxxxx
```

### Akamata 本体を入手

Akamata はフレームワーク本体と `akamata` CLI を同じリポジトリに含んでいます。

```
git clone https://github.com/yourorg/Akamata
cd Akamata
zig build cli
```

ビルドが成功すると `zig-out/bin/akamata` ができます。 `PATH` に追加しておくと このあとが楽です:

```
# 一時的に通す場合
export PATH="$PWD/zig-out/bin:$PATH"

# 永続化 (zsh の場合)
echo 'export PATH="'"$PWD"'/zig-out/bin:$PATH"' >> ~/.zshrc
```

確認:

```
akamata help
```

```
Usage: akamata <command> [args]

Commands:
  init <name> [--target=native|workers|containers|both]
    Scaffold a new Akamata app.
  build [--workers|--containers]
    Build the current app (native by default).
  ...
```

**ハマりどころ:** `command not found: akamata` が出る場合、`PATH` の追加を `.zshrc` に書いた後に新しいターミナルウィンドウを開く必要があります。または `./zig-out/bin/akamata help` のようにフルパスで呼んでも構いません。

## 1. プロジェクトの雛形を作る

### この章のゴール

- `akamata init` で `mytodo` プロジェクトの雛形を生成
- できたディレクトリ構成を理解する

### コマンド

Akamata リポジトリの **外側** の作業ディレクトリに移動してから:

```
cd ~/projects          # 適当な作業ディレクトリ
akamata init mytodo --target=both
```

`--target=both` は「ネイティブバイナリと Cloudflare Workers の両方で動く構成」を意味します。

### 期待出力

```
Created mytodo/

Next steps:
  cd mytodo
  zig build run          # native dev server
```

## ディレクトリ構成

```
cd mytodo
tree -a -L 2
```

```
mytodo/
├── .gitignore
├── README.md
├── build.zig                # ビルド設定
├── build.zig.zon           # 依存ライブラリ宣言（今は Akamata 本体だけ）
├── deploy/
│   ├── wrangler.toml      # Cloudflare Workers の設定
│   └── worker/
│       └── index.mjs       # Workers の JS ホスト（wasm をロードする）
└── src/
    ├── main.zig           # アプリ本体（これから書き換える）
    └── worker.zig         # Workers エントリーポイント
```

## 各ファイルの役割

| ファイル                                 | 役割                                                            |
|--------------------------------------|---------------------------------------------------------------|
| <code>build.zig</code>               | Zig のビルド設定。コマンドラインの <code>-Dbackend=workers</code> などはここが解釈する |
| <code>build.zig.zon</code>           | パッケージ依存。Akamata 本体への参照を含む                                     |
| <code>src/main.zig</code>            | ネイティブ起動時のエントリーポイント。今回はここに 9 割書く                               |
| <code>src/worker.zig</code>          | Workers 起動時のエントリーポイント。 <code>main.zig</code> の関数を呼び出すだけ       |
| <code>deploy/wrangler.toml</code>    | D1 binding や環境変数の定義                                           |
| <code>deploy/worker/index.mjs</code> | Workers の JavaScript シム。wasm をロードして D1 をブリッジする                |

覚えること: ハンドラを書き足すときに編集するのは `src/main.zig` だけです。

## 2. 雛形を起動して中身を見る

### この章のゴール

- 生成された雛形をそのままビルド・起動する
- curl で動作確認
- src/main.zig の構造を一通り読む

### ビルドして起動

```
zig build run
```

初回ビルドは1〜2分かかります。Akamata 本体と SQLite (約 200K 行の C) を一緒に コンパイルするためです。2 回目以降は数秒で終わります。

期待出力:

```
info: mytodo listening on :8080
info: akamata listening on http://0.0.0.0:8080/
```

## 動作確認

別ターミナルを開いて:

```
curl -sS http://127.0.0.1:8080/
```

```
{"name":"mytodo","endpoints":{"health":"GET /health","list":"GET /notes","create":"POST /not
```

雛形はデフォルトで `Note` という別モデルが入っています。これからこれを `Todo` に作り直します。

サーバを止めるには `Ctrl-C`。

## src/main.zig をざっと読む

`src/main.zig` を開くと、次のセクションに分かれています:

```
// ===== Model =====
pub const Note = struct { ... }; // ← データモデル
const Notes = am.model.repo(Note); // ← Note の CRUD ヘルパー

// ===== App state =====
pub const State = struct { db: am.db.Db }; // ← 全ハンドラで共有する状態
pub const all_models = [_]am.model.TableDef{ ... };

// ===== Handlers =====
fn index(c: *Ctx) !void { ... } // ← 各 HTTP ハンドラ
fn createNote(c: *Ctx) !void { ... }

// ===== Wiring =====
pub fn registerRoutes(app: *am.App(State)) !void { ... }
pub fn buildState(alloc: std.mem.Allocator) !State { ... }
pub fn main(...) !void { ... } // ← エントリーポイント
```

## Zig コードの読み方の基礎 (Zig 未経験者向け)

```
const std = @import("std");
const am = @import("akamata");
```

`const x = value;` で**変更不可な変数**を宣言。 `@import` でモジュール (パッケージ) を読み込みます。  
`am` は Akamata の略です。

```
pub const Todo = struct {
    id: ?i64 = null,
    title: []const u8,
};
```

- `pub` = この struct を外部から見えるように公開
- `?i64` = 「`i64` または `null`」を意味する optional 型
- `[]const u8` = `u8` のスライス。文字列はこの型を使います
- `= null` / `= ""` = デフォルト値

```
fn hello(c: *Ctx) !void {
    try c.text("Hello");
}
```

- `*Ctx` = `Ctx` 型へのポインタ (リクエストごとに 1 つ作られる)
- `!void` = 「成功なら `void`、失敗ならエラー」を返す関数
- `try` = 「エラーなら呼び出し元に伝播」する糖衣構文 (Rust の `?` に近い)

これだけ覚えれば 9 割読めます。

## 3. Todo モデルを定義する

### この章のゴール

- `Note` モデルを `Todo` モデルに置き換える
- 優先度 (`priority`) と完了フラグ (`completed`) を追加
- インデックスとデフォルト値も設定

### モデル設計

| カラム                     | Zig 型                     | SQL 型               | 説明            |
|-------------------------|---------------------------|---------------------|---------------|
| <code>id</code>         | <code>?i64 = null</code>  | INTEGER PRIMARY KEY | 主キー (自動採番)    |
| <code>title</code>      | <code>[]const u8</code>   | TEXT NOT NULL       | タイトル          |
| <code>priority</code>   | <code>i32 = 3</code>      | INTEGER NOT NULL    | 1=高, 2=中, 3=低 |
| <code>completed</code>  | <code>bool = false</code> | INTEGER NOT NULL    | 完了フラグ         |
| <code>created_at</code> | <code>?i64 = null</code>  | INTEGER (UNIX秒)     | 作成日時          |

### 書き換える

`src/main.zig` の `Note` セクションを次の `Todo` モデルに置き換えます (元の `Note` 関連は 丸ごと削除して構いません):

```
// ===== Model =====

pub const Todo = struct {
    id: ?i64 = null,
    title: []const u8,
    priority: i32 = 3,
    completed: bool = false,
    created_at: ?i64 = null,

    pub const __schema = .{
        .table = "todos",
        .primary_key = "id",
        .indexes = .{
            // 「未完了 → 優先度高い順 → 作成日時降順」の典型クエリを高速化
            .{ .{ "completed", "priority", "created_at" }, .index },
        },
        .defaults = .{
            .created_at = "unixepoch()",
        },
        .validates = .{
            .title = .{ am.model.rule.required, am.model.rule.max_len(200) },
            .priority = .{ am.model.rule.range(1, 3) },
        },
    };
};

const Todos = am.model.repo(Todo);
```

## \_\_schema の各セクションの意味

| キー          | 何をするか                                               |
|-------------|-----------------------------------------------------|
| table       | SQL のテーブル名。省略すると todos (struct 名 + "s") になる         |
| primary_key | 主キーのフィールド名。デフォルト "id"                               |
| indexes     | インデックス定義。unique か index を末尾に置く                      |
| defaults    | SQL の DEFAULT (expr) 句。unixepoch() は SQLite の組み込み関数 |
| validates   | 後で紹介するバリデーション                                       |

## all\_models の更新

少し下にある all\_models も更新します:

```
pub const all_models = [_]am.model.TableDef{
    am.model.tableDef(Todo),
};
```

**覚えること:** 新しいモデルを追加したら all\_models に必ず追記。自動マイグレーションがここを見て CREATE TABLE を発行します。

## 4. 自動マイグレーションで DB を作る

### この章のゴール

- 起動時に自動でテーブルが作成されることを理解
- 既存 DB に対して差分で `ALTER TABLE` が走ることを確認

雛形の `buildState` 関数はすでに自動マイグレーションを呼んでいます。Note モデルから Todo に変えただけで、再ビルド + 再起動するだけで切り替わります。

### 旧 DB を消してから起動

```
rm -f mytodo.db          # 古い Note 用 DB を消す
zig build run
```

ログを確認:

```
info: mytodo listening on :8080
```

エラーが出なければ OK。テーブルができたか SQLite CLI で確認 (任意):

```
sqlite3 mytodo.db ".schema"
```

期待出力:

```
CREATE TABLE schema_migrations (...);
CREATE TABLE todos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  priority INTEGER NOT NULL,
  completed INTEGER NOT NULL,
  created_at INTEGER DEFAULT (unixepoch())
) STRICT;
CREATE INDEX todos_completed_priority_created_at_idx ON todos (completed, priority, created_at);
```

### 差分マイグレーションの例 (後でやってもよい)

たとえばあとから `notes: []const u8 = ""` フィールドを追加すると、再起動時に:

```
warn: migrate.diff found drift, applying: ALTER TABLE todos ADD COLUMN notes TEXT NOT NULL
```

のようにログが出て自動で `ALTER TABLE` が走ります。DROP COLUMN は危険なので 自動では実行されず、警告だけ出ます。

**仕組み:** `am.model.migrate.diff()` が `PRAGMA table_info()` の結果とモデルの `__schema` を比較し、差分の `ALTER TABLE ADD COLUMN` / `CREATE INDEX` を生成します。



## 5. CRUD ハンドラを実装する

---

### この章のゴール

- 5 つのハンドラ (`list`, `create`, `show`, `update`, `delete`) を書く
- `Todos.find/all/where/create/save/delete` の使い方を覚える

`src/main.zig` の Handlers セクションをまるごと次の内容に置き換えます:

```
// ===== Handlers =====

const Ctx = am.Context(State);

fn index(c: *Ctx) !void {
    // HTML UI は後の章で追加。今はエンドポイント一覧を JSON で返す
    try c.json(.{
        .name = "mytodo",
        .endpoints = .{
            .list = "GET /api/todos",
            .create = "POST /api/todos { title, priority? }",
            .show = "GET /api/todos/:id",
            .update = "PUT /api/todos/:id { title?, priority?, completed? }",
            .delete = "DELETE /api/todos/:id",
        },
    }, 200);
}

fn health(c: *Ctx) !void {
    // DB に届くか確かめる軽い ping
    var stmt = c.db().prepare("SELECT 1") catch return c.serverError("db unavailable");
    defer stmt.deinit();
    _ = stmt.step() catch return c.serverError("db unavailable");
    try c.json(.{ .status = "ok" }, 200);
}

fn listTodos(c: *Ctx) !void {
    // 未完了 → 優先度高 → 新しい順 (インデックスが効くクエリ)
    const rows = try Todos.queryRaw(c.db(), c.arena,
        "SELECT id, title, priority, completed, created_at FROM todos " ++
        "ORDER BY completed ASC, priority ASC, created_at DESC LIMIT 200",
        .{});
    try c.json(.{ .todos = rows }, 200);
}

fn createTodo(c: *Ctx) !void {
    // 入力 JSON をパースし、Todo モデルのバリデータも走らせる。
    // 失敗した場合は c.input が自動で 400 / 422 を返す。
    const todo = (try c.input(Todo)) orelse return;
    const created = try Todos.create(c.db(), c.arena, todo);
    try c.json(created, 201);
}

fn showTodo(c: *Ctx) !void {
    const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
    const t = (try Todos.find(c.db(), c.arena, id)) orelse return c.notFound();
    try c.json(t, 200);
}

fn updateTodo(c: *Ctx) !void {
    const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
    var t = (try Todos.find(c.db(), c.arena, id)) orelse return c.notFound();

    // 部分更新を受け取るために、すべて optional な struct で別途 JSON を読む

```

```

const Patch = struct {
  title: ?[]const u8 = null,
  priority: ?i32 = null,
  completed: ?bool = null,
};
const patch = c.req.json(Patch) catch return c.badRequest("invalid JSON");
if (patch.title) |v| t.title = try c.arena.dupe(u8, v);
if (patch.priority) |v| t.priority = v;
if (patch.completed) |v| t.completed = v;

try Todos.save(c.db(), c.arena, &t);
try c.json(t, 200);
}

fn deleteTodo(c: *Ctx) !void {
  const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
  try Todos.delete(c.db(), id);
  try c.json(.{ .deleted = id }, 200);
}

```

## コードの読みどころ

### `c.db()` と `c.arena`

- `c.db()` = `State.db` への近道 (= `c.state().db`)。SQL を投げる先
- `c.arena` = リクエスト単位のメモリプール。このリクエストの間だけ生きるメモリ

レスポンスを返したらアリーナごと開放されるので、`free` を呼ぶ必要はありません。

### `c.input(Todo)`

```
const todo = (try c.input(Todo)) orelse return;
```

これだけで:

1. JSON をパース
2. `Todo.__schema.validates` のルールを実行
3. 失敗時は 400 (パース失敗) / 422 (バリデーション失敗) を自動で返す
4. 成功時は `Todo` 値を返す

`orelse return` で「失敗ならハンドラを抜ける (レスポンスはすでに書き込み済み)」というパターンになります。

### `queryRaw` を使った場合と `all` の違い

`Todos.all()` だと `ORDER BY id DESC` 固定です。今回は「未完了 → 優先度」というビジネスロジック特有の並びにしたいので `queryRaw` (生 SQL のエスケープハッチ) を使いました。

## ルート登録

`registerRoutes` を次に置き換えます:

```
pub fn registerRoutes(app: *am.App(State)) !void {
    _ = try app.useAll(am.mw.recover(State));
    _ = try app.useAll(am.mw.logger(State));

    _ = try app.get("/", index);
    _ = try app.get("/health", health);
    _ = try app.get("/api/todos", listTodos);
    _ = try app.post("/api/todos", createTodo);
    _ = try app.get("/api/todos/:id", showTodo);
    _ = try app.put("/api/todos/:id", updateTodo);
    _ = try app.delete("/api/todos/:id", deleteTodo);
}
```

## 動かしてみる

```
zig build run
```

別ターミナルで:

```
# 1. 作成
curl -sS -X POST -H 'content-type: application/json' \
  -d '{"title":"買い物","priority":1}' \
  http://127.0.0.1:8080/api/todos
```

期待:

```
{"id":1,"title":"買い物","priority":1,"completed":false,"created_at":1779999999}
```

```
# 2. 一覧
curl -sS http://127.0.0.1:8080/api/todos
```

```
{"todos":[{"id":1,"title":"買い物","priority":1,"completed":false,"created_at":1779999999}]}
```

```
# 3. 完了にする
curl -sS -X PUT -H 'content-type: application/json' \
  -d '{"completed":true}' \
  http://127.0.0.1:8080/api/todos/1
```

```
{"id":1,"title":"買い物","priority":1,"completed":true,"created_at":1779999999}
```

```
# 4. 削除
curl -sS -X DELETE http://127.0.0.1:8080/api/todos/1
```

```
{"deleted":1}
```

## 6. バリデーションを追加する

### この章のゴール

- 不正な入力を弾けるか確認
- カスタムバリデータも書ける

### 標準ルールの動作確認

すでに `Todo.__schema.validates` で:

- `title` は必須 + 最大 200 文字
- `priority` は 1 ~ 3

を宣言しました。実際に弾かれるか試します。

```
# title が空 → 422
curl -sS -w "\nHTTP %{http_code}\n" -X POST \
  -H 'content-type: application/json' \
  -d '{"title":"","priority":1}' \
  http://127.0.0.1:8080/api/todos
```

期待:

```
{"error_kind":"validation","errors":[{"field":"title","rule":"required","message":"is required"}]}
HTTP 422
```

```
# priority が範囲外 → 422
curl -sS -w "\nHTTP %{http_code}\n" -X POST \
  -H 'content-type: application/json' \
  -d '{"title":"x","priority":99}' \
  http://127.0.0.1:8080/api/todos
```

期待:

```
{"error_kind":"validation","errors":[{"field":"priority","rule":"range","message":"must be between 1 and 3"}]}
HTTP 422
```

```
# JSON が壊れている → 400
curl -sS -w "\nHTTP %{http_code}\n" -X POST \
  -H 'content-type: application/json' \
  -d 'not json' \
  http://127.0.0.1:8080/api/todos
```

期待:

```
{"error_kind":"bad_request","message":"invalid JSON body"}
HTTP 400
```

## カスタムバリデータ

たとえば「タイトルに `<script>` を含まない」というルールを足したい場合:

```
fn noScriptTag(value: []const u8, _: std.mem.Allocator) ?[]const u8 {
    if (std.mem.indexOf(u8, value, "<script>") != null) {
        return "must not contain <script>";
    }
    return null;
}

pub const Todo = struct {
    // ...省略...
    pub const __schema = .{
        // ...省略...
        .validates = .{
            .title = .{
                am.model.rule.required,
                am.model.rule.max_len(200),
                am.model.rule.custom(noScriptTag),
            },
            .priority = .{ am.model.rule.range(1, 3) },
        },
    };
};
```

- 戻り値 `null` = OK
- 戻り値 文字列 = エラーメッセージ (そのまま返される)

整数フィールド向けには `am.model.rule.customInt(fn)` があります。

---

## 7. HTML UI を embedFile で同梱する

### この章のゴール

- ブラウザでアクセスして UI から CRUD 操作できる
- HTML/CSS/JS をすべて wasm/native バイナリに焼き込む
- `Accept` ヘッダで JSON と HTML を出し分ける

### index.html を用意

`src/index.html` を新規作成:

```

<!doctype html>
<html lang="ja">
<head>
<meta charset="utf-8">
<title>mytodo</title>
<style>
  body { font-family: system-ui, sans-serif; max-width: 640px; margin: 2rem auto; padding: 0 1rem; }
  h1 { margin: 0 0 1rem; }
  form { display: flex; gap: 0.5rem; margin-bottom: 1.5rem; }
  input[type="text"] { flex: 1; padding: 0.4rem 0.6rem; font: inherit; border: 1px solid #999; border-radius: 0.4rem; }
  select, button { padding: 0.4rem 0.6rem; font: inherit; border: 1px solid #999; border-radius: 0.4rem; }
  button[type="submit"] { background: #2563eb; color: #fff; border-color: #2563eb; cursor: pointer; }
  ul { list-style: none; padding: 0; }
  li { display: flex; align-items: center; gap: 0.5rem; padding: 0.5rem; border-bottom: 1px solid #ccc; }
  li.done .title { text-decoration: line-through; color: #888; }
  .pri { width: 1.4rem; height: 1.4rem; border-radius: 50%; display: inline-block; }
  .pri-1 { background: #ef4444; }
  .pri-2 { background: #f59e0b; }
  .pri-3 { background: #9ca3af; }
  .title { flex: 1; }
  .del { color: #b91c1c; background: transparent; border: none; cursor: pointer; }
  .err { color: #b91c1c; font-size: 0.9rem; min-height: 1.2rem; }
</style>
</head>
<body>
<h1>mytodo</h1>
<form id="f">
  <input type="text" id="title" placeholder="新しいやること" required maxlength="200">
  <select id="priority">
    <option value="1">高</option>
    <option value="2">中</option>
    <option value="3" selected>低</option>
  </select>
  <button type="submit">追加</button>
</form>
<div id="err" class="err"></div>
<ul id="list"></ul>

<script>
const $ = (id) => document.getElementById(id);
const api = async (method, path, body) => {
  const r = await fetch(path, {
    method,
    headers: body ? { "content-type": "application/json" } : {},
    body: body ? JSON.stringify(body) : null,
  });
  if (!r.ok) {
    const j = await r.json().catch(() => ({ message: r.statusText }));
    throw new Error(j.message || JSON.stringify(j.errors));
  }
  return r.json();
};

async function refresh() {
  try {

```

```

$("err").textContent = "";
const { todos } = await api("GET", "/api/todos");
$("list").innerHTML = todos.map(t => `
  <li class="${t.completed ? "done" : ""}" data-id="${t.id}">
    <input type="checkbox" ${t.completed ? "checked" : ""} class="chk">
    <span class="pri pri-${t.priority}" title="priority ${t.priority}"></span>
    <span class="title">${escapeHtml(t.title)}</span>
    <button class="del">削除</button>
  </li>
`).join("");
[...$("list").children].forEach((li, i) => {
  const id = parseInt(li.dataset.id, 10);
  li.querySelector(".chk").onchange = (e) =>
    api("PUT", `/api/todos/${id}`, { completed: e.target.checked }).then(refresh);
  li.querySelector(".del").onclick = () =>
    api("DELETE", `/api/todos/${id}`).then(refresh);
});
} catch (e) {
  $("err").textContent = e.message;
}
}

function escapeHtml(s) {
  return s.replace(/[<>"']/g, c => ({ "&":"&amp;","<":"&lt;",">":"&gt;","'":"&quot;","'":"'&quot;","'":"'&quot;"}
}

$("f").onsubmit = async (e) => {
  e.preventDefault();
  try {
    $("err").textContent = "";
    await api("POST", "/api/todos", {
      title: $("title").value,
      priority: parseInt($("priority").value, 10),
    });
    $("title").value = "";
    refresh();
  } catch (err) {
    $("err").textContent = err.message;
  }
};

refresh();
</script>
</body>
</html>

```

## src/main.zig に embedFile を追加

ファイル上部の `const Notes = ...` の下あたり (Handlers セクションの直上) に追加:

```
const index_html = @embedFile("index.html");
```

`@embedFile` はコンパイル時にファイル全体をバイナリ文字列として `[]const u8` で埋め込みます。実行時にファイルを読まないの、デプロイ先にファイルを置く必要がありません。



## index ハンドラを HTML/JSON 分岐に

すでに書いた `index` 関数を次のように変更:

```
fn index(c: *Ctx) !void {
    // ブラウザは Accept: text/html を送ってくる → HTML UI を返す
    // curl やプログラムは Accept: */* なので JSON を返す
    const accept = c.req.header("accept") orelse "";
    if (std.mem.indexOf(u8, accept, "text/html") != null) {
        return c.html(index_html);
    }
    try c.json(.{
        .name = "mytodo",
        .endpoints = .{
            .list = "GET /api/todos",
            .create = "POST /api/todos { title, priority? }",
            .show = "GET /api/todos/:id",
            .update = "PUT /api/todos/:id { title?, priority?, completed? }",
            .delete = "DELETE /api/todos/:id",
        },
    }, 200);
}
```

## 起動して確認

```
zig build run
```

ブラウザで <http://127.0.0.1:8080/> を開くと、Todo の追加・完了・削除がすべて UI からできるはずです。

```
mytodo
[      新しいやること      ] [低 v] [追加]

○ ● 買い物                  [削除]
✓ ● 報告書を書く            [削除]
```

(色付きの丸が優先度、チェックボックスが完了フラグ、× が削除)

## Accept ヘッダの確認

```
# curl のデフォルトは Accept: */* → JSON
curl -sS http://127.0.0.1:8080/

# ブラウザを模倣 → HTML
curl -sS -H 'Accept: text/html' http://127.0.0.1:8080/ | head -3
```

## 8. ルーティングとミドルウェアを整える

### この章のゴール

- すでに使っている `recover` / `logger` の意味を理解
- 本番向けに `accessLog` (構造化ログ) と `metrics` を追加
- API パスを `basePath` でまとめる

### ミドルウェアの順序

`registerRoutes` の先頭で `useAll` した順に **全ルート** に適用されます。順序が重要です:

```
_ = try app.useAll(am.mw.recover(State)); // 1. panic を 500 にしてつかむ
_ = try app.useAll(am.mw.logger(State)); // 2. ログ
```

`recover` を一番外側に置くのは「中で panic が起きてもユーザに 500 を返せるよう 最後の砦を作る」ためです。

### 観測強化: requestId + accessLog + metrics

`registerRoutes` の冒頭を次のように拡張します:

```
var metrics_counters: am.MetricsCounters = .{};

pub fn registerRoutes(app: *am.App(State)) !void {
  _ = try app.useAll(am.mw.recover(State));
  _ = try app.useAll(am.mw.requestId(State)); // X-Request-ID 自動付与
  _ = try app.useAll(am.mw.accessLog(State, .json)); // 構造化 1 行ログ
  _ = try app.useAll(am.mw.metrics(State, &metrics_counters));

  _ = try app.get("/", index);
  _ = try app.get("/health", health);
  _ = try app.get("/metrics", am.mw.metricsHandler(State, &metrics_counters));

  _ = try app.get("/api/todos", listTodos);
  _ = try app.post("/api/todos", createTodo);
  _ = try app.get("/api/todos/:id", showTodo);
  _ = try app.put("/api/todos/:id", updateTodo);
  _ = try app.delete("/api/todos/:id", deleteTodo);
}
```

**重要:** `metrics_counters` は **モジュールレベルの var** にする必要があります (関数内 var だと寿命が切れる)。 `registerRoutes` の外、 `State` 定義の近くに書きます。

### 確認

`zig build run` で起動して、いくつかリクエストを投げてから:

```
curl -sS http://127.0.0.1:8080/metrics | head -20
```

```
# HELP akamata_requests_total Total HTTP requests served.
# TYPE akamata_requests_total counter
akamata_requests_total 12
# HELP akamata_requests_in_flight Requests currently being processed.
# TYPE akamata_requests_in_flight gauge
akamata_requests_in_flight 1
# HELP akamata_requests_by_status Requests broken down by HTTP status class.
# TYPE akamata_requests_by_status counter
akamata_requests_by_status{class="1xx"} 0
akamata_requests_by_status{class="2xx"} 10
akamata_requests_by_status{class="3xx"} 0
akamata_requests_by_status{class="4xx"} 2
akamata_requests_by_status{class="5xx"} 0
...
```

アクセスログは `zig build run` のターミナルに JSON 1 行ずつ出ます:

```
{"ts_unix_us":1779999999000000,"req_id":"a64d6f73-2b0e-4ad1-9aa3-8c0f4f2c5d6e","ip":"-","met
```

`req_id` は `X-Request-ID` レスポンスヘッダにも入るので、フロントエンドのエラー レポートとサーバ ログを突き合わせるのに使えます。

詳細は [docs/observability.md](https://docs.cloudflare.com/observability.md) を参照。

---

## 9. Cloudflare Workers + D1 にデプロイする

---

### この章のゴール

- ローカルで動いた同じコードを Workers wasm にビルド
- D1 データベースを作成し、Akamata 経由で自動マイグレーション
- 本番 URL でブラウザから動作確認

### 9.1 ローカルで Workers モードを試す

まず `wrangler dev --local` でローカル Miniflare 上で wasm を動かします。

```
# Workers 用 wasm をビルド
zig build -Dbackend=workers -Doptimize=ReleaseSmall

# wrangler dev で起動
cd deploy
wrangler dev --local --port 18080
```

期待出力 (一部):

```
☁️ wrangler 4.93.1
Your Worker has access to the following bindings:
Binding              Resource              local
env.DB (mytodo)      D1 Database
[wrangler:info] Ready on http://localhost:18080
```

別ターミナルから:

```
curl -sS http://127.0.0.1:18080/health
```

```
{"status":"ok"}
```

`/api/todos` への POST も同じく動きます。ローカルの SQLite ファイルではなく、Miniflare のシミュレートされた D1 に書き込まれます。

どうやって動いているか: `src/worker.zig` がエクスポートする `handle_fetch` という wasm 関数を `deploy/worker/index.mjs` が JSPI (JavaScript Promise Integration) 経由で呼び出しています。D1 binding の `env.DB.prepare(sql).run()` を Zig 側から あたかも同期関数のように呼べる仕組みです。詳しくは [docs/db-backends.md](#) 参照。

`Ctrl-C` で停止。

## 9.2 本番 Cloudflare にデプロイ

```
# プロジェクトルートに戻る
cd ..

# 一発デプロイ (D1 作成 + マイグレーション + デプロイ)
akamata deploy --workers \
  --config=deploy/wrangler.toml \
  --migrate=<(/zig-out/bin/mytodo --print-schema)
```

このコマンドの裏で起こること:

1. `wrangler.toml` の `[[d1_databases]]` ブロックを読む
2. `database_id` がプレースホルダなら `wrangler d1 create` を実行、UUID を書き戻す
3. `mytodo --print-schema` を実行 (現在のモデルから DDL SQL を生成)
4. その SQL を `wrangler d1 execute --remote` で D1 に流す
5. `zig build -Dbackend=workers -Doptimize=ReleaseSmall`
6. `wrangler deploy`

期待出力 (一部):

```
==> akamata: provisioning D1 "mytodo" (database_id is placeholder)
==> akamata: resolved D1 "mytodo" (id=abcdef12-3456-7890-...)
==> akamata: wrote new database_id back to deploy/wrangler.toml
==> akamata: applying ... to remote D1 "mytodo"
==> akamata: building wasm (ReleaseSmall)
==> akamata: wrangler deploy
...
Uploaded mytodo (2.33 sec)
Deployed mytodo triggers (0.96 sec)
https://mytodo.<your-subdomain>.workers.dev
```

最後の URL を**ブラウザで開く**と、ローカルと同じ UI が表示されます。D1 はサーバレスでスケールするので、ここから本番運用が始められます。

## 9.3 デプロイ後の確認

```
URL=https://mytodo.<your-subdomain>.workers.dev

curl -sS $URL/health
curl -sS -X POST -H 'content-type: application/json' \
  -d '{"title":"first prod todo","priority":1}' \
  $URL/api/todos
curl -sS $URL/api/todos
```

すべて 200 OK が返ってくれば成功です。

### トラブルシューティング:

- `Did you mean ...?` のエラー → CLI のサブコマンドのタイポ
- `D1_EXEC_ERROR` → スキーマと既存テーブルの不整合。 `wrangler d1 execute mytodo --remote --command="DROP TABLE todos"` で初期化してから再度 `akamata deploy --migrate`

## 10. 本番稼働のための観測 (metrics + ログ)

### この章のゴール

- 本番で起きていることを把握できる
- PromQL を使えなくても問題なし — `/metrics` 単体でデバッグできる

### 10.1 メトリクスの読み方

`/metrics` エンドポイントは Prometheus テキスト形式で生のカウンターを返します:

```
akamata_requests_total 5891
akamata_requests_in_flight 0
akamata_requests_by_status{class="2xx"} 5700
akamata_requests_by_status{class="4xx"} 188
akamata_requests_by_status{class="5xx"} 3
akamata_requests_by_method{method="GET"} 4502
akamata_requests_by_method{method="POST"} 1280
akamata_request_latency_seconds_bucket{le="0.0001"} 4810
akamata_request_latency_seconds_bucket{le="0.001"} 5722
akamata_request_latency_seconds_bucket{le="0.01"} 5891
akamata_request_latency_seconds_bucket{le="+Inf"} 5891
akamata_request_latency_seconds_count 5891
akamata_request_latency_seconds_sum 0.872413
akamata_process_resident_memory_bytes 2621440
akamata_process_uptime_seconds 1234
```

これだけで:

- **平均レイテンシ** =  $\text{sum} / \text{count} = 0.872 / 5891 \approx 148 \mu\text{s}$
- **エラー率** =  $(4xx + 5xx) / \text{total} = 191 / 5891 \approx 3.2 \%$
- **uptime** が想定通りか確認

がわかります。

## 10.2 Prometheus + Grafana

本格的に運用するなら `prometheus.yml` に:

```
scrape_configs:
  - job_name: mytodo
    metrics_path: /metrics
    scrape_interval: 15s
    static_configs:
      - targets: ['mytodo.example.com:8080']
```

PromQL 例:

```
# 1 分あたり req/sec
rate(akamata_requests_total[1m])

# P99 レイテンシ
histogram_quantile(0.99, sum by (le) (rate(akamata_request_latency_seconds_bucket[5m])))

# 5xx エラー率
sum(rate(akamata_requests_by_status{class="5xx"}[5m]))
/ sum(rate(akamata_requests_total[5m]))
```

詳しくは [docs/observability.md](#) を参照。

## 10.3 アクセスログを別ファイルに

`am.mw.accessLog(State, .json)` は標準エラーに出力します。本番でファイルに保存するなら起動時にリダイレクト:

```
./zig-out/bin/mytodo 2> >(tee -a /var/log/mytodo.log >&2)
```

Workers の場合は `wrangler tail` か Logpush で Cloudflare Analytics に流せます。

## 11. よくある問題とデバッグ

### `error: cannot find file` (build 時)

- `src/index.html` を作り忘れているか、`@embedFile` のパスタイプ。
- パスは `src/main.zig` からの相対パス

### `wrangler dev` で `SuspendError`

```
SuspendError: trying to suspend without WebAssembly.promising
```

ワーカーホスト側 `deploy/worker/index.mjs` の `WebAssembly.promising` 設定が古い場合に起こります。`akamata init` で生成された雛形を使っていれば問題ないはず。過去のテンプレートを手で書き換えた場合は再生成してください。

### サーバが固まる/反応しない

ハンドラ内で `while (true)` などの無限ループを書いていないか確認。Akamata は 1 リクエスト 1 スレッドなので、無限ループは 1 スレッドを占有するだけですが、慢性的に増えるとスループットが落ちます。

### `BodyTooLarge` がログに出る

クライアントが大きすぎる POST を送っています。デフォルトのボディ上限は 8 MB。大きくしたければ `ServeOptions.max_body_bytes` を増やす:

```
try app.serve({ .port = 8080, .max_body_bytes = 32 * 1024 * 1024 });
```

### マイグレーションが失敗する

- ローカル: `rm mytodo.db` で DB を消してリセット
- Workers + D1: `wrangler d1 execute mytodo --remote --command="DROP TABLE todos"` のあと再デプロイ

### ホットリロードが効かない

Zig はコンパイル言語なので、`Ctrl-C` → `zig build run` の手動再起動が必要です。ファイル変更で自動再起動したい場合は `entr` 等のツールを併用:

```
ls src/*.zig | entr -r zig build run
```

## 12. 次のステップ

このチュートリアルで触ったのは Akamata の基本機能だけです。さらに踏み込むなら:

### より大きなアプリ

- `examples/mobus/` — 26 エンドポイント、JWT 認証、フレンド機能、WS チャット、FCM push の本格アプリ
- `examples/chat/` — Durable Object SQLite + WebSocket の組み合わせ

### 個別トピック

- [docs/handbook.md](#) — 15 分で全機能を概観 (このチュートリアルの圧縮版)
- [docs/db-backends.md](#) — SQLite / Turso / D1 の挙動差と JSPI 仕組み
- [docs/observability.md](#) — 本番運用のメトリクス・ログ詳細
- [docs/benchmarks.md](#) / [benchmarks-long-run.md](#) — 性能特性
- [docs/architecture.md](#) — フレームワーク内部設計と本番ハードニング

### モデル層を深掘り

- リレーション (`has_many` / `belongs_to`)
- Eager loading (`am.model.preload.hasMany`) — N+1 問題回避
- カスタムバリデーター
- カラム名リネーム (`__schema.columns`) — Zig は camelCase、SQL は snake\_case の使い分け
- 複数モデル + foreign key

### コミュニティ

- 質問・バグ報告は GitHub Issues
- 改善案は Pull Request 歓迎

お疲れさまでした。Akamata で楽しい開発を!